Evaluation of Machine Learning Models for Continuous User Authentication Using Keyboard and Mouse Behavioral Biometrics

Carlos T. Bautista III *, David Emmanuel L. Lacsao*, Marjorie S. Millama*, Precious Julia Ablir*,
Philip Virgil B. Astillo †, Godwin S. Monserate †
*University of San Carlos, Student Member
Cebu City, Cebu, Philippines
25400054@usc.edu.ph, 21102046@usc.edu.ph, 25200084@usc.edu.ph, 25104850@usc.edu.ph
†University of San Carlos, Member
Cebu City, Cebu, Philippines

***Abstract*——-Network security is crucial to data confidentiality, integrity, and control. Traditional security measures, such as passwords, passkeys, and PINs, are commonly used to ensure safe and secure access to entrusted users. However, due to the rapid digitalization, intruders learn to steal user data and information that leads to unwanted access. As a result, more advanced features are used to strengthen network security. Instead of traditional keys, behavioral biometrics such as keystroke dynamics, mouse patterns, touchscreen interactions, and many more are used to train systems to identify and authenticate users. In modeling, however, different algorithms can modify how models perform in user authentication. This study evaluates the machine learning algorithms, Random Forest and XG Boost Algorithm, used in the proposed behavioral biometrics-based continuous network authentication system that analyzes keystroke dynamics and mouse movement patterns. This study aims to analyze which ML algorithm better authenticates normal user behavior and detects anomalies that may indicate insider threats or unauthorized access, providing an additional security layer beyond traditional password authentication.**

I. Introduction

In the emerging growth of technology, digital access has consequently led to unwanted intrusion. Data theft, political interference, and ransomware extortion are some of the intrusion agenda items. To address this security issue, especially in private networks, authenticating user identity and protecting computer systems from unauthorized access have become increasingly important. Conventional authentication models, such as PINs and passwords, have been vulnerable to social engineering attacks. As a result, researchers explore smarter and more adaptable approaches to authenticate users.

Behavioral biometric patterns, such as keyboard typing and mouse movement patterns, are unique for every user despite the similarity of tasks assigned in a shared network. This can be used for user identification, authentication, and verification [1]. Similar to how individuals write papers and perform specific tasks, behavioral biometrics can be authenticated through the use of user-based learning algorithms. Authentication algorithms have existed to study behavioral patterns of users for several applications such as IoT devices security, healthcare systems, and surveillance [2]. However, applications in educational institutions for network authentication is not yet a standard practice. This opens doors for possible intrusion such as account hacking and potential identity theft.

Behavioral biometrics has been used as a unique user identifier in network intrusion detection systems. Several studies explore different algorithms to model user keyboard and mouse patterns to detect authorized users from intruders. This study investigates the performance of two machine learning models, namely Random Forest and XGBoost, in classifying users from merged behavioral biometric data. Different train-validation ratios and feature subset sizes are evaluated to determine how feature selection and model choice affect classification accuracy and validation performance. Gini importance is used as a selection method to determine top features that contribute to reducing impurity across all trees in a model. By comparing these algorithms using Gini importance for feature selection under identical experimental conditions, the study aims to identify the most suitable model for real-time behavioral authentication applications.

II. Review of Related Literature

A. Keyboard Patterns for Behavioral Biometrics

Behavioral biometrics refer to user profiles created from patterns in how individuals interact with devices, such as typing habits, mouse movements, and touch gestures [3]. These behavioral patterns are used by systems as a reference baseline for identifying possible intrusions from unauthorized individuals within a network. During network access, the system continuously monitors and compares the user's current behavior with the stored baseline profile to confirm identity and identify suspicious activities. This process is commonly known as anomaly detection in related studies [4] [5].

Keystroke dynamics, also referred to as typing patterns, is a type of behavioral biometric that examines the way individuals use a keyboard while typing [6]. Unlike traditional authentication methods such as passwords, fingerprint scanning, and

facial recognition, keystroke dynamics focuses on typing characteristics, including speed, rhythm, and timing. This makes it an effective method for continuous and unobtrusive user authentication [7].

An early study [8] showed that typing rhythms could effectively differentiate users with a reasonable level of accuracy. The researchers introduced the concept that timing data between keystrokes, such as digraph latency or the interval between two consecutive key presses, can function as a dependable biometric identifier. They also suggested that each person possesses a distinct typing rhythm, making it a natural option for computer security applications. This study became the basis for future developments in behavioral authentication systems [9] [10] [11].

Recent research has enhanced typing pattern authentication through the application of machine learning and deep learning techniques [12] [13]. In 2025, a study introduced a deep learning–based authentication system that utilized Siamese neural networks to evaluate typing behaviors [14]. The findings indicated that integrating keystroke dynamics with adaptive thresholding improved the accuracy of detecting unauthorized users, particularly in highly secure environments. In addition, other studies have shown that modern artificial intelligence methods increase the reliability and security of behavioral biometric systems [15] [16].

In general, keystroke dynamics focuses on users' typing behaviors, including factors such as timing, speed, and key pressure. This study continuously observes the typing patterns of authorized users and uses them as a baseline for the intrusion detection system (IDS). Machine learning techniques are applied to develop the IDS, allowing the system to establish normal behavior patterns and identify anomalies within the network.

B. Mouse Dynamics for Behavioral Biometrics

Behavioral biometrics using mouse dynamics, also known as pointer behavior, has become a promising method for continuous authentication because it verifies user identity through natural interactions with computer devices in a non-intrusive manner. Researchers have explored various techniques for modeling, extracting, and analyzing mouse behavior, which are generally classified into feature-based, trajectory-based, temporal, and hybrid machine learning methods [17].

Various studies have demonstrated that mouse patterns can serve as a behavioral biometric for authentication and cybersecurity purposes. Researchers observed that individuals exhibit unique mouse usage behaviors, such as differences in cursor movement speed, direction, scrolling habits, and drag-and-drop actions [18]. These behavioral characteristics can be utilized to recognize legitimate users and identify unauthorized access attempts. Mouse dynamics is also viewed as a non-intrusive and cost-effective approach since it operates in the background while users interact with the computer normally [19].

Other studies have examined the application of machine learning and deep learning techniques in analyzing mouse behavior. Researchers utilized deep learning models to enhance the accuracy of user identification through mouse movement patterns [20], while other studies developed authentication systems capable of distinguishing genuine users from impostors based on mouse dynamics [21]. Furthermore, continuous authentication through mouse clickstream analysis has been shown to support real-time monitoring and strengthen security protection [22].

Researchers have also investigated the integration of mouse dynamics with other behavioral biometrics, including keystroke patterns. A study by Sukumar Mondal and Patrick Bours (2014) reported that combining multiple behavioral biometric methods resulted in better authentication performance compared to relying on a single technique alone [23].

These studies demonstrated that mouse behavior can enhance traditional password-based authentication methods and contribute to stronger computer security. The findings revealed that information collected from an individual's mouse behavioral biometrics can successfully identify legitimate users and help minimize unauthorized access. This paper further supports the use of mouse behavioral biometrics together with keystroke dynamics as a dependable and efficient approach for continuous authentication and intrusion detection systems.

C. Comparison of XGBoost and Random Forest as Machine Learning Algorithms for User Authentication

XGBoost and Random Forest are commonly used machine learning algorithms in user authentication systems. These algorithms help classify user behavior patterns such as keystroke dynamics and mouse movements to determine whether a user is authorized or suspicious.

Random Forest works by combining multiple decision trees to improve classification accuracy and reduce overfitting. It is known for its stability, fast processing, and ability to handle large datasets effectively. Random Forest provides reliable results for classification tasks and is widely used in behavioral biometric systems[24].

XGBoost is a boosting algorithm that improves prediction performance by correcting errors from previous models. XGBoost is designed for high accuracy and efficient performance, making it suitable for behavioral biometric analysis and intrusion detection[25].

Studies comparing the two algorithms show that both are effective in user authentication. However, XGBoost often achieves higher accuracy, while Random Forest is faster and easier to interpret. These algorithms are important in improving the security and reliability of behavioral biometric authentication systems.

Overall, both XGBoost and Random Forest provide strong performance in behavioral biometric authentication. Their ability to analyze user behavior patterns helps improve the accuracy, reliability, and security of continuous authentication systems, making them valuable tools in detecting unauthorized access and protecting user accounts. In this paper, the two algorithms are compared to determine which model performs

better in analyzing keystroke dynamics and mouse behavioral biometrics for user authentication and intrusion detection.

III. Data and methods

A. Research Design

The focus of this study is using behavioral biometrics for the classification of users identified based on the data collected on each user. The users were identified through a python script that detects them continuously using keyboard and mouse behaviors for the classification. The system continuously monitored the computer usage, and a machine learning algorithm runs in the background that is trained using the previously collected data of each user. The goal is to detect the users on host computers in real time, and the server sends a network block command to disable a specific user from being connected to the same network.

During the data collection phase, keyboard and mouse features were gathered for each person. Raw activities were converted into behavioral features that were used for training, and each of these features was grouped into 5-second windows. The system generates predictions continuously when the script is activated on the computer. The algorithm continuously predicts the current user engaging with the computer, and each prediction represents the detected user

The system consists of multiple Host computers and only one Server computer that oversees all host computers. The host computer monitors the users locally and generates the predictions. The predictions are sent to the server computer periodically. TCP sockets were used for communication between host and server, and JSON format was used for the transfer of data. The server checks whether the current predicted user is allowed or blocked. Once it detects a blocked user on a specific host computer, a block command is sent, and the Wi-Fi network on that computer is immediately disabled.

B. Description of Datasets

This paper analyzes keystroke dynamics, combined with mouse patterns, for intrusion detection system modeling. A total of 64 features for keystroke dynamics are collected per user. The features include key inputs: a-z, 0-9, and other keys, namely, dot, comma, question mark, colon, control, shift, enter, backspace, and space bar. It also includes features, namely:

- **Words Per Minute (WPM).** WPM is the measure of a user's typing speed, calculated based on the number of words typed within one minute.
- **Average Hold Time (`avg_hold_time`).** Average hold time refers to the average duration a key remains pressed before being released during typing.
- **Average Digraph Latency (`avg_digraph_latency`).** Average digraph latency is the average time interval between two consecutive keystrokes, representing the transition speed between keys.
- **Average Press-to-Press Latency (`avg_pp_latency`).** Average press-to-press latency measures the time difference between successive key press events.
- **Backspace Rate (`backspace_rate`).** Backspace rate indicates the frequency of backspace key usage during typing, which may reflect typing corrections and user behavior patterns.
- **Pause Count (`pause_count`).** Pause count refers to the number of detected pauses or interruptions while the user is typing.
- **Burst Ratio (`burst_ratio`).** Burst ratio measures the intensity of continuous typing activity by analyzing typing bursts separated by pauses.
- **Average Mouse X Position (`avg_mouse_x`).** Average mouse X position represents the mean horizontal cursor position recorded during user interaction.
- **Average Mouse Y Position (`avg_mouse_y`).** Average mouse Y position represents the mean vertical cursor position recorded during user interaction.
- **Average Mouse Speed (`avg_mouse_speed`).** Average mouse speed measures the average velocity of cursor movement across the screen.
- **Average Mouse Acceleration (`avg_mouse_accel`).** Average mouse acceleration measures the rate of change in cursor speed during movement.
- **Mouse Path Efficiency (`mouse_path_efficiency`).** Mouse path efficiency evaluates how directly the cursor moves from one point to another relative to the total path traveled.
- **Total Mouse Distance (`mouse_total_distance`).** Total mouse distance refers to the cumulative distance traveled by the cursor during interaction.
- **Mouse Direction Changes (`mouse_direction_changes`).** Mouse direction changes measure the frequency at which the cursor changes movement direction.
- **Mouse Left Click (`mouse_left_click`).** Left click count represents the number of left mouse button clicks performed by the user.
- **Mouse Right Click (`mouse_right_click`).** Right click count represents the number of right mouse button clicks performed by the user.
- **Average Click Dwell Time (`avg_click_dwell_time`).** Average click dwell time measures the average duration between mouse button press and release events.
- **Scroll Events (`scroll_events`).** Scroll events refer to the number of scrolling actions performed using the mouse wheel.
- **Average Scroll Speed (`avg_scroll_speed`).** Average scroll speed is the average speed of scrolling activity during user interaction.

C. Data Collection Method

Four participants are instructed to perform controlled and random sessions for the data collection, which are the researchers themselves. The duration of the data collection lasted for 7 days. Two sessions are conducted for each day: random and controlled sessions, with each lasting for 10 minutes.

During these sessions, a python script runs in the background to monitor the keyboard and mouse behavior. After each session, the data is compiled into one CSV and once each user has finished all of the 7-day data collections, the researchers compile all 4 CSVs into one final CSV that is used for training the classification model.

Controlled sessions are when each user answers typing tasks that are done on the google classroom platform. Each day has specific tasks that are the same for all users, which they are required to do each day. These tasks involve a repetitive typing task where they are to duplicate the same sentence on each line that should manually be written 10 times. Another task is free typing, where each user answers questions like what the things they did that day, their favorite sport, and other personal reflective questions. Each user is required to stay on the platform for the entire duration of the controlled session. The purpose of this is to collect similar, consistent behavioral patterns performed by each user.

Random sessions involve the user doing anything they want, like browsing, gaming, social media usage, research activities, and watching videos. The purpose of this is to collect real-world behavior of each user that they would naturally do in their everyday activities. These would let the algorithm have more personal behavioral patterns for each user involved. The users also stayed on their computer for the entire duration of the random session.

Keyboard and mouse events were continuously recorded during each session when the python script was activated. Keyboard events like key frequencies, backspace rate, words per minute, key presses, key release, and other keyboard features were collected. For the mouse events, features including mouse speed, acceleration, average positions, click behavior, path efficiency, and other features were being recorded. Since each user used computers with different screen sizes and hardware specifications, normalization was applied to the mouse-related features to maintain consistency across different devices.

D. Data Cleaning

All the collected data were processed into cleaned data using the Gini Importance. Feature selection was performed using the Random Forest and XG Boost classifier through the Mean Decrease in Gini Impurity method. The Gini impurity of a node is computed as:

$$Gini(D) = 1 - \sum_{i=1}^{C} p_i^2$$

where D represents the raw dataset at a node, C is the total number of data classes, and p_i is the probability of class i.

For each feature f, the decrease in Gini impurity after splitting was calculated using:

$$\Delta Gini(f) = Gini(D_p) - \left(\frac{N_{left}}{N_p} Gini(D_{left}) + \frac{N_{right}}{N_p} Gini(D_{right}) \right)$$

where D_p is the parent node, D_{left} and D_{right} are the child nodes, N_p is the number of samples in the parent node, and N_{left} and N_{right} are the number of samples in the left and right child nodes, respectively.

The Gini importance of all the features was measured by getting the average of the impurity reduction contributed by each feature across all decision trees in the model:

$$FI(f) = \frac{1}{T} \sum_{t=1}^{T} \sum_{n \in t} \Delta Gini_n(f)$$

where $FI(f)$ is the feature importance score of feature f, T is the total number of decision trees, and $\Delta Gini_n(f)$ represents the impurity reduction contributed by feature f at node n.

The features were ranked according to their computed importance scores:

$$TopFeatures = \text{sort}(FI(f), \text{descending})$$

The top k features with the highest importance scores were then selected for model training:

$$F_{top} = \{f_1, f_2, ..., f_k\}$$

where F_{top} represents the selected subset of top-ranked features.

E. Data Modeling and Analysis

Training and validation accuracies are calculated and compared to measure the model's classification efficiency. Python is used for scripting in data collection, cleaning, and modeling. All CSV files are organized and compiled for the data processing. To determine the optimal number of key features and splitting ratio for training and testing, a dataset consisting of 10, 15, 20, and 25 features and splitting ratios of 90:10, 80:20, 70:30, and 60:40 is evaluated.

Overfitting analysis will also be conducted to compare the two models based on the relationship between training and validation accuracies. Overfitting is calculated using:

$$\text{Overfitting Value} = A_{\text{train}} - A_{\text{validation}}$$

where:

- A_{train} is the training accuracy
- $A_{\text{validation}}$ is the validation accuracy

Since the data collection set-up is limited to only 7 sessions, it is expected that the accuracy levels are not as high as a trained robust model. However, the analysis remains sufficient for evaluating relative model performance and identifying trends in overfitting behavior.

IV. Results

A. Ranked Features of Keystroke Dynamics and Mouse Patterns Data

The datasets are cleaned using Gini impurities and ranked on the basis of their importance values. The list is summarized in Fig. 1. Among the features used in the data collection, the top list primarily includes avg_hold_time, avg_click_dwell_time, mouse_avg_y,

wpm, avg_pp_latency, avg_digraph_latency, mouse_avg_x, mouse_avg_speed, burst_ratio, and mouse_total_distance.

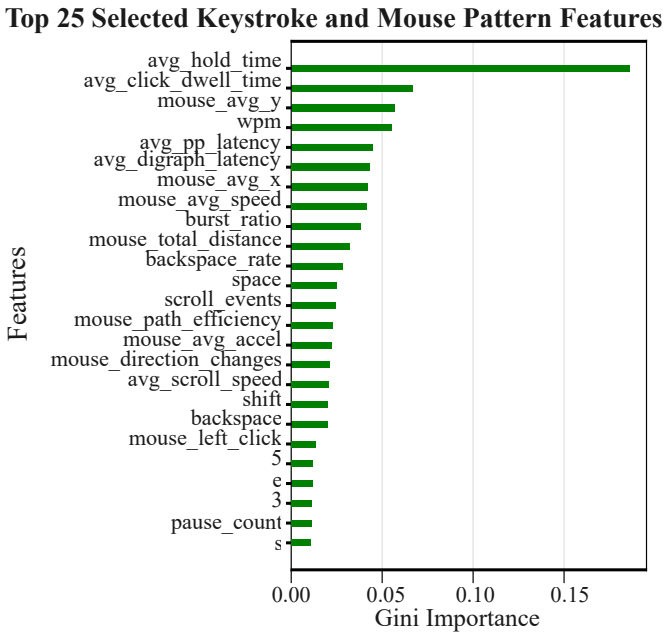

Fig. 1: Top Keystroke and Mouse Features selected using Gini Importance.

B. Data Accuracy

Model accuracy is measured during both training and validation across all feature split ratios for the two models using Random Forest and XG Boost algorithms. Based on the results, an average of 72.94% training accuracy with 61.99% validation accuracy is achieved using the Random Forest algorithm. This is slightly smaller than the achieved training and validation accuracies of the XG Boost Algorithm, which are 77.35% and 63.25%, respectively. The results show that XG Boost generally provides better classification performance. However, the higher training accuracy than the validation accuracy also suggests a greater tendency toward overfitting. In contrast, Random Forest shows more consistent behavior between training and validation performance, indicating better generalization capability.

C. Overfitting Analysis

Overfitting values are calculated to measure how far validation accuracy is from the training accuracy for the model across all configurations. Based on the results, a 0.1806 overfitting value or 18.06% overfitting rate was measured for the model using XG Boost, which is quite higher than the model using Random Forest, having a 0.1498 overfitting value or 14.98%overfitting rate. This indicates that although XG Boost achieved higher accuracy, it also exhibited a greater tendency to overfit the training data. In comparison, the Random Forest model demonstrated more stable generalization performance, as reflected by the smaller gap between training and validation accuracies.

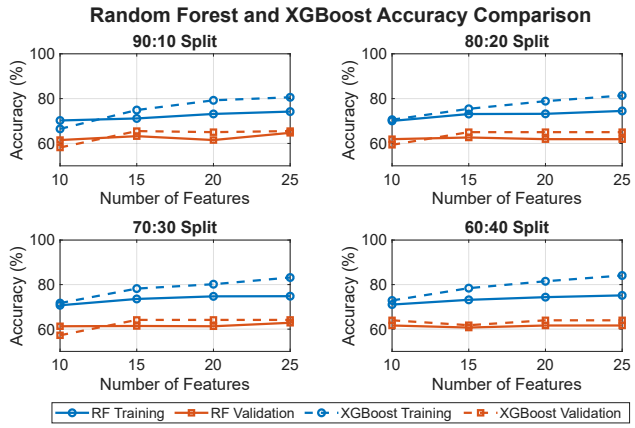

Fig. 2: Training and Validation Accuracy of the Two Models

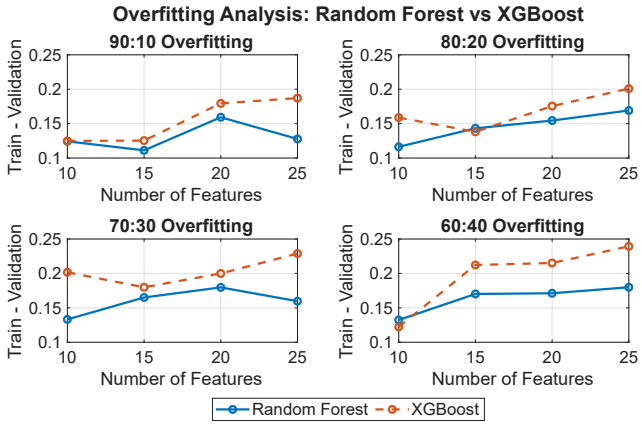

Fig. 3: Overfitting Analysis of the Two Models

D. Confusion Matrix for the Two Models

The confusion matrix is used to evaluate the classification performance of the Random Forest-based model by comparing the predicted classifications with the true user labels. Optimal split ratio and number of features are used for each algorithm. It provides a detailed assessment of model performance beyond overall accuracy by quantifying correct and incorrect classifications in terms of true positives, true negatives, false positives, and false negatives. Through this analysis, the ability of the model to correctly distinguish legitimate users from intruders can be examined. The results show that out of the four users, four are predicted correctly. Out of 514 predictions, 373 were predicted correctly for User1, 268 for User2, 324 for User3, and 302 for User4.

For the XG Boost-based model, four out of four users are also identified correctly. However, prediction values were much less than the values of the Random Forest-based model. Out of 514 predictions of the true labels, 270 were predicted for User1, 181 for User2, 211 for User3, and 201 for User4. These results indicate that although the XG Boost model was capable of correctly distinguishing all users, its classification consistency and confidence were comparatively lower than

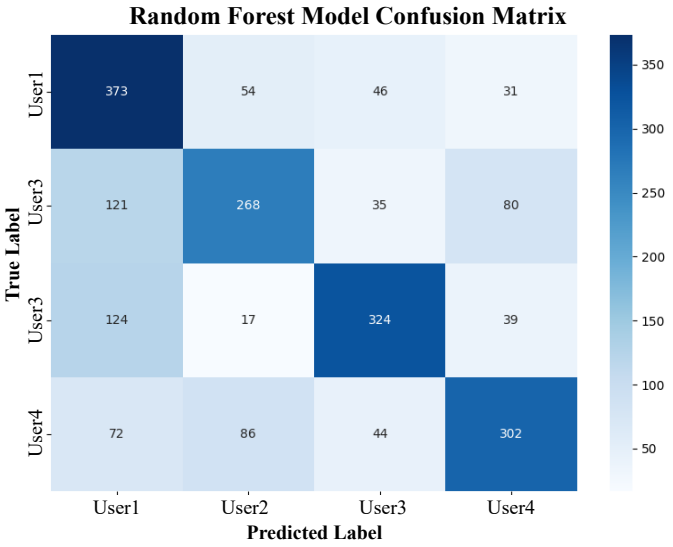

Fig. 4: Confusion Matrix of the Model using Random Forest

those of the Random Forest model. This behavior may be associated with the higher overfitting tendency observed in the XG Boost model, which affected the general classification capability.

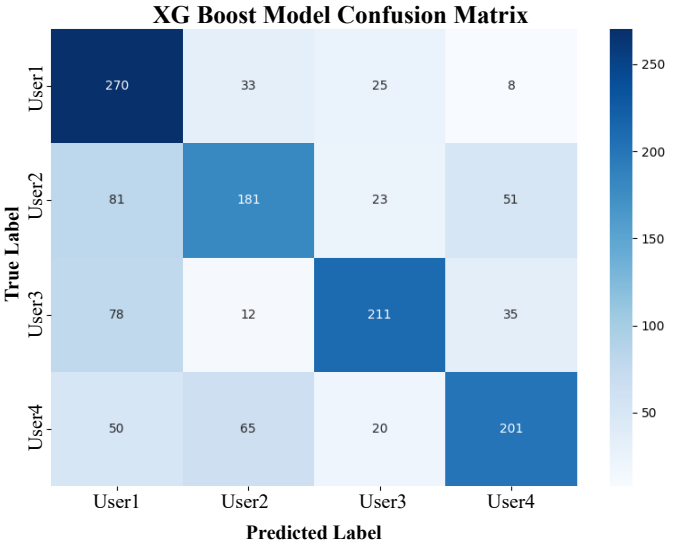

Fig. 5: Confusion Matrix of the Model using XG Boost

V. Discussion

Based on the results presented above, the model using Random Forest generally performs better than XG Boost. Despite the higher training and validation accuracy, the overfitting value is comparatively higher for the model using XG Boost than Random Forest. As plotted in the confusion matrix, the Random Forest algorithm has higher predicting values relative to the true labels. This suggests that although XGBoost is able to fit the training data well, it is more prone to capturing noise and dataset-specific patterns rather than robust underlying trends.

In contrast, the Random Forest model maintains a more balanced performance between training and validation sets, which reflects better stability and generalization capability. As observed in the confusion matrix, the Random Forest algorithm also produces higher and more consistent correct classification counts across all users relative to the true labels. This indicates that its predictions are more evenly distributed and less biased toward overfit patterns.

In general, the Random Forest model demonstrates stronger reliability for this limited dataset, achieving a more favorable trade-off between accuracy and generalization, whereas XG-Boost, despite its strong learning capacity with higher training and validation accuracy, appears to overfit more significantly under the same conditions.

VI. Summary

This study generally evaluates two algorithms, namely, Random Forest and XG Boost, for user authentication in a shared network. The model explores behavioral biometrics: keystroke dynamics and mouse patterns as unique identifiers to establish security in a school laboratory network. The features are cleaned and selected using Gini impurity and ranked using the Gini Importance formula. The training and validation process was configured using different numbers of features (10, 15, 20, and 25) and split ratios (90:10, 80:20, 70:30, and 60:40).

Based on the results, Random Forest demonstrates better generalization capability than XG Boost. It has balanced performance in terms of accuracy and overfitting rate, which in turn produces better predictive results across the true labels for each user. Although XG Boost achieved 77.35% training accuracy rate with 63.25% validation accuracy rate, which is slightly higher than the achieved 72.94% training accuracy rate and 62.99% validation rate of the Random Forest-based model, the overfitting value (18.06%) is still comparatively higher than that of Random Forest (14.98%). Basically, Random Forest performs better than XG Boost in general.

Future work will explore alternative machine learning algorithms that may provide improved generalization performance compared to both Random Forest and XGBoost. In particular, models such as deep learning architectures or hybrid ensemble methods may be investigated to better capture complex patterns in the data while reducing overfitting tendencies. Additionally, it is highly recommended to strengthen the dataset by adding more data collection sessions with significant classes for more optimized modeling and training. Other feature selection methods are also suggested to be explored for more precise cleaning of datasets.

Acknowledgment

We would like to express our sincere gratitude to our research advisers, Philip Virgil Astillo and Godwin Monserate, for the guidance, support, and encouragement throughout the

completion of this study. Their knowledge and suggestions greatly helped improve our research.

We would also like to thank our teachers, classmates, and the University of San Carlos for providing the resources and opportunity to conduct this research. Special thanks are also given to our family, and friends for their continuous support, understanding, and motivation during the development of this study. And to everyone who contributed directly or indirectly, thank you.

Finally, we thank God for the guidance and wisdom that led to the success of this research paper.

REFERENCES

[1] Das, B.B., Reddy, C.V., Matha, U. et al. Multimodal biometric authentication systems: exploring EEG and signature. Cogn Neurodyn 20, 17 (2026). https://doi.org/10.1007/s11571-025-10389-w

[2] S. Essahraui and I. Lamaakal, "A Comprehensive Survey of TinyML-Based Biometric Recognition for IoT Edge Devices," in IEEE Internet of Things Journal, vol. 13, no. 6, pp. 10564-10588, 15 March15, 2026, doi: 10.1109/JIOT.2026.3656932.

[3] C. MacAlpine, "The future of user authentication: A guide to behavioral biometrics," Jan. 6, 2026. [Online]. Available: https://expertinsights.com/user-auth/a-guide-to-behavioral-biometrics

[4] S. Parhizkari, "Anomaly detection—Recent advances, AI and ML perspectives and applications," 2023, doi: 10.5772/intechopen.112733.

[5] M. Khan, "The importance of combined user and data behavior analysis in anomaly detection," 2022.

[6] A. Lo, V. H. Ayma, and J. Gutierrez-Cardenas, "A comparison of authentication methods via keystroke dynamics," in *Proc. IEEE Eng. Int. Res. Conf. (EIRCON)*, Lima, Peru, 2020, pp. 1–4, doi: 10.1109/EIRCON51178.2020.9253751.

[7] F. Monrose and A. Rubin, "Keystroke dynamics as a biometric for authentication," *Future Gener. Comput. Syst.*, vol. 16, pp. 351–359, 2000, doi: 10.1016/S0167-739X(99)00059-X.

[8] S. Hussain, "Behavioral biometrics and continuous authentication in cybersecurity systems," 2025, doi: 10.13140/RG.2.2.35971.41763.

[9] O. L. Finnegan *et al.*, "The utility of behavioral biometrics in user authentication and demographic characteristic detection: A scoping review," *Syst. Rev.*, vol. 13, no. 1, p. 61, 2024, doi: 10.1186/s13643-024-02451-1.

[10] K. Shende, M. Marbate, P. Hore, S. Wazalwar, and N. Wyawahare, "A review on behavioral biometric authentication system," in *Proc. 12th Int. Conf. Emerg. Trends Eng. Technol. Signal Inf. Process. (ICETET-SIP)*, Nagpur, India, 2025, pp. 1–6, doi: 10.1109/ICETET-SIP64213.2025.11156378.

[11] A. Budžys, O. Kurasova, and V. Medvedev, "Integrating deep learning and data fusion for advanced keystroke dynamics authentication," *Comput. Standards Interfaces*, vol. 92, p. 103931, 2025, doi: 10.1016/j.csi.2024.103931.

[12] A. Muralidharan, A. Eaman, and E. Hassan, "A comparative analysis of machine learning models for behavioral biometric authentication using keystroke dynamics," *Procedia Comput. Sci.*, vol. 265, pp. 116–123, 2025, doi: 10.1016/j.procs.2025.07.163.

[13] V. Medvedev, A. Budžys, and O. Kurasova, "A decision-making framework for user authentication using keystroke dynamics," *Comput. Secur.*, vol. 155, p. 104494, 2025, doi: 10.1016/j.cose.2025.104494.

[14] U. Javid and E. Kollwitz, "AI-based behavioral biometrics for next-generation digital identity verification," 2025, doi: 10.13140/RG.2.2.32452.13448.

[15] S. Ravan and P. Dhotre, "Secure authentication using biometric and behavioral analysis," in *ICT Analysis and Applications*, 2025, pp. 129–140.

[16] S. Amin and C. Iorio, "A review of several keystroke dynamics methods," 2025, doi: 10.48550/arXiv.2502.16177.

[17] Jorgensen, Z., & Yu, T. (2011). On mouse dynamics as a behavioral biometric for authentication. In Proceedings of the 6th International Symposium on Information, Computer and Communications Security, ASIACCS 2011 (pp. 476-482). (Proceedings of the 6th International Symposium on Information, Computer and Communications Security, ASIACCS 2011). Association for Computing Machinery. https://doi.org/10.1145/1966913.1966983

[18] Khan, S., Devlen, C., Manno, M., & Hou, D. (2022). Mouse dynamics behavioral biometrics: A survey. arXiv. https://arxiv.org/abs/2208.09061

[19] Jorgensen, Z., & Yu, T. (2011). On mouse dynamics as a behavioral biometric for authentication. Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security.

[20] Antal, M., & Egyed-Zsigmond, E. (2020). Mouse dynamics based user recognition using deep learning. Acta Universitatis Sapientiae, Informatica, 12(1), 39–50. https://doi.org/10.2478/ausi-2020-0003

[21] Salman, O. A., Hameed, S. M. (2018). User authentication via mouse dynamics. Iraqi Journal of Science, 59(2B), 963–968.

[22] Almalki, S., Chatterjee, P., Roy, K. (2023). Continuous authentication using mouse clickstream data analysis. arXiv. https://arxiv.org/abs/2312.00802

[23] Mondal, S., & Bours, P. (2014). User identification and authentication using multi-modal behavioral biometrics. Computers & Security, 43, 77–89. https://doi.org/10.1016/j.cose.2014.03.005

[24] Breiman, L. (2001). Random Forests. Machine Learning Journal, 45(1), 5–32.

[25] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.